Create Sections

1.Import Libraries

2.Load Dataset

3.Understand Dataset

4.Data Cleaning

5.Exploratory Data Analysis

6.Complaint Analysis

7.Statistical Testing

8.Observations

STEP:1 Understand The Dataset

1.1 Import Dataset

In [1]:
```
import pandas as pd

df = pd.read_csv('311_Service_Requests_from_2010_to_Present.csv',
                 encoding = 'latin1',
                 low_memory =False)

df.head()
```

Out[1]:

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Descriptor
0	32310363	12/31/2015 11:59:45 PM	01/01/2016 12:55:15 AM	NYPD	New York City Police Department	Noise - Street/Sidewalk	Loud Music/Party
1	32309934	12/31/2015 11:59:44 PM	01/01/2016 01:26:57 AM	NYPD	New York City Police Department	Blocked Driveway	No Access
2	32309159	12/31/2015 11:59:29 PM	01/01/2016 04:51:03 AM	NYPD	New York City Police Department	Blocked Driveway	No Access
3	32305098	12/31/2015 11:57:46 PM	01/01/2016 07:43:13 AM	NYPD	New York City Police Department	Illegal Parking	Commercial Overnight Parking
4	32306529	12/31/2015 11:56:58 PM	01/01/2016 03:24:42 AM	NYPD	New York City Police Department	Illegal Parking	Blocked Sidewalk

5 rows × 53 columns

1.2 Visualize

In [2]:
```
df.head()
df.sample(5)
```

Out[2]:

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Descri
270626	30526321	05/01/2015 09:47:08 AM	05/01/2015 10:47:15 AM	NYPD	New York City Police Department	Blocked Driveway	Pa Ac
30797	32098031	11/30/2015 07:09:02 PM	11/30/2015 09:56:18 PM	NYPD	New York City Police Department	Illegal Parking	Po Par Viola
290408	30370600	04/10/2015 10:25:56 PM	04/11/2015 04:42:08 AM	NYPD	New York City Police Department	Noise - Street/Sidewalk	L Tal
153877	31272679	08/10/2015 12:11:18 AM	08/10/2015 07:57:13 AM	NYPD	New York City Police Department	Noise - Street/Sidewalk	L Music/F
232281	30764289	06/03/2015 11:13:09 AM	06/03/2015 11:45:25 AM	NYPD	New York City Police Department	Derelict Vehicle	Lic

5 rows × 53 columns

1.3 Print Columns

In [3]:
```
df.columns
```

Out[3]:
```
Index(['Unique Key', 'Created Date', 'Closed Date', 'Agency', 'Agency Name',
       'Complaint Type', 'Descriptor', 'Location Type', 'Incident Zip',
       'Incident Address', 'Street Name', 'Cross Street 1', 'Cross Street 2',
       'Intersection Street 1', 'Intersection Street 2', 'Address Type',
       'City', 'Landmark', 'Facility Type', 'Status', 'Due Date',
       'Resolution Description', 'Resolution Action Updated Date',
       'Community Board', 'Borough', 'X Coordinate (State Plane)',
       'Y Coordinate (State Plane)', 'Park Facility Name', 'Park Borough',
       'School Name', 'School Number', 'School Region', 'School Code',
       'School Phone Number', 'School Address', 'School City', 'School State',
       'School Zip', 'School Not Found', 'School or Citywide Complaint',
       'Vehicle Type', 'Taxi Company Borough', 'Taxi Pick Up Location',
       'Bridge Highway Name', 'Bridge Highway Direction', 'Road Ramp',
       'Bridge Highway Segment', 'Garage Lot Name', 'Ferry Direction',
       'Ferry Terminal Name', 'Latitude', 'Longitude', 'Location'],
      dtype='str')
```

1.4 Identify Shape

In [4]:
```
df.shape
```

Out[4]:
```
(364558, 53)
```

1.5 Identify Variables With Null Values

In [5]:
```
df.isnull().sum()
```

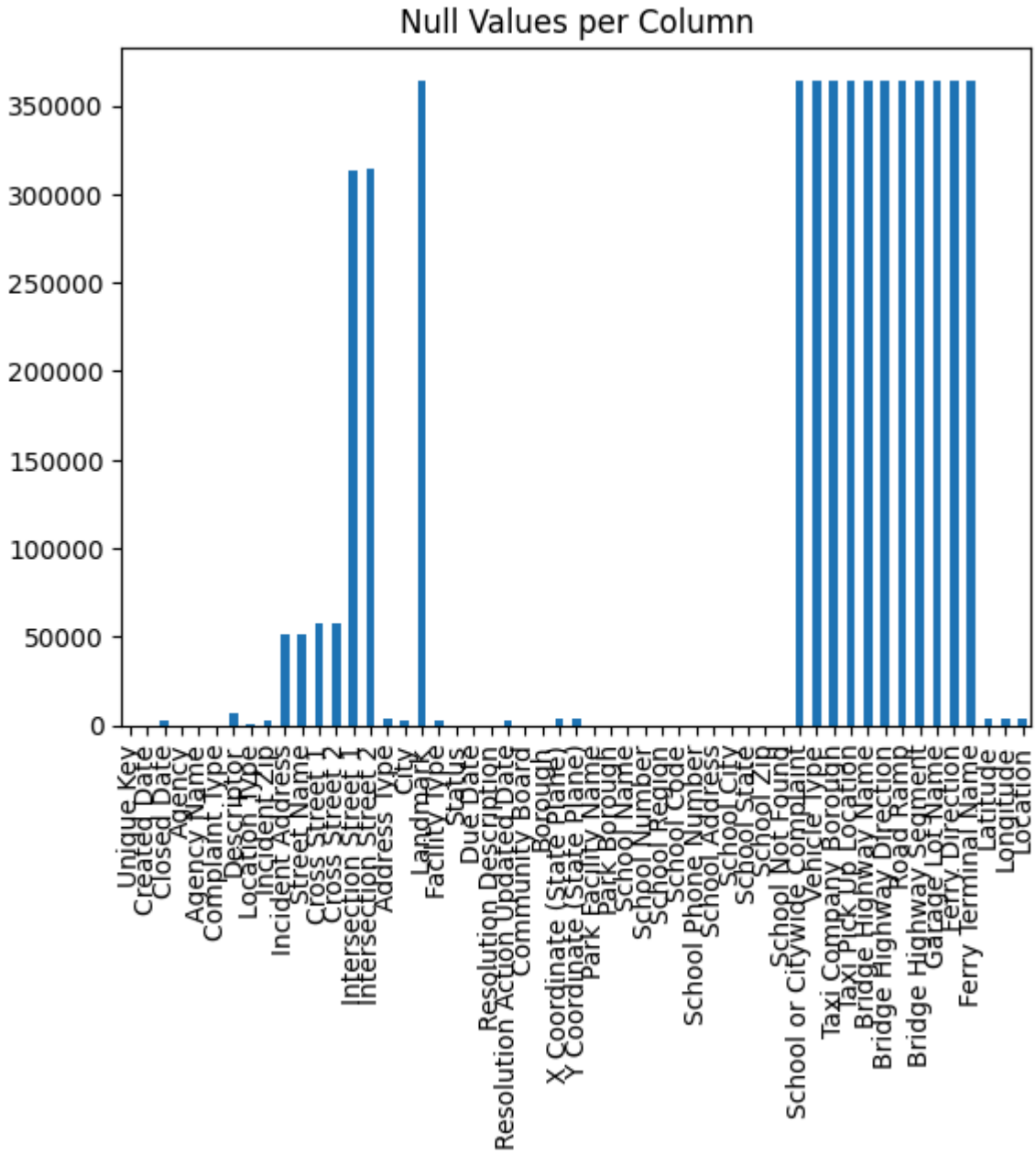
Out[5]:
```
Unique Key                           0
Created Date                         0
Closed Date                       2381
Agency                               0
Agency Name                          0
Complaint Type                       0
Descriptor                        6501
Location Type                      133
Incident Zip                      2998
Incident Address                 51699
Street Name                      51699
Cross Street 1                   57188
Cross Street 2                   57805
Intersection Street 1           313438
Intersection Street 2           314046
Address Type                      3252
City                              2997
Landmark                        364183
Facility Type                     2389
Status                               0
Due Date                             3
Resolution Description               0
Resolution Action Updated Date    2402
Community Board                      0
Borough                              0
X Coordinate (State Plane)        4030
Y Coordinate (State Plane)        4030
Park Facility Name                   0
Park Borough                         0
School Name                          0
School Number                        0
School Region                        1
School Code                          1
School Phone Number                  0
School Address                       0
School City                          0
School State                         0
School Zip                           1
School Not Found                     0
School or Citywide Complaint    364558
Vehicle Type                    364558
Taxi Company Borough            364558
Taxi Pick Up Location           364558
Bridge Highway Name             364261
Bridge Highway Direction        364261
Road Ramp                       364296
Bridge Highway Segment          364296
Garage Lot Name                 364558
Ferry Direction                 364557
Ferry Terminal Name             364556
Latitude                          4030
Longitude                         4030
Location                          4030
dtype: int64
```

Step 2: Basic EDA

2.1 Frequency Plot To Show Null Value in Each Column

In [7]:
```
import matplotlib.pyplot as plt

df.isnull().sum().plot(kind='bar')
plt.title("Null Values per Column")
plt.xticks(rotation=90)
plt.show()
```



2.1 Missing Value Treatment

2.1.1 Remove Closed Date Null

In [8]:
```
df= df[df['Closed Date'].notnull()]
```

2.3 Analyze Date Column

In [9]:
```
df['Created Date'] = pd.to_datetime(
    df['Created Date'],
    format='%m/%d/%Y %I:%M:%S %p'
)

df['Closed Date'] = pd.to_datetime(
    df['Closed Date'],
    format='%m/%d/%Y %I:%M:%S %p'
)
```

2.3 Remove Wrong Timeline

In [10]:
```
df = df[df['Closed Date'] >= df['Created Date']]
```

2.3.1 Calculate the time elapsed in closed and creation date

In [11]:
```
df['Request_Closing_Time'] = df['Closed Date'] - df['Created Date']
```

2.3.2 Convert the calculated date to seconds to get a better representation

In [12]:
```
df['Request_Closing_Time'] = df['Request_Closing_Time'].dt.total_seconds()
```

2.3.3 View the descriptive statistics for the newly created column

In [13]:
```
df['Request_Closing_Time'].describe()
```

Out[13]:
```
count    3.621770e+05
mean     1.511330e+04
std      2.110255e+04
min      6.100000e+01
25%      4.533000e+03
50%      9.616000e+03
75%      1.887800e+04
max      2.134342e+06
Name: Request_Closing_Time, dtype: float64
```

2.3.4 Check the number of null values in the Complaint_Type and City columns

In [14]:
```
df['City']= df['Borough']

df[['Complaint Type', 'City']].isnull().sum()
```

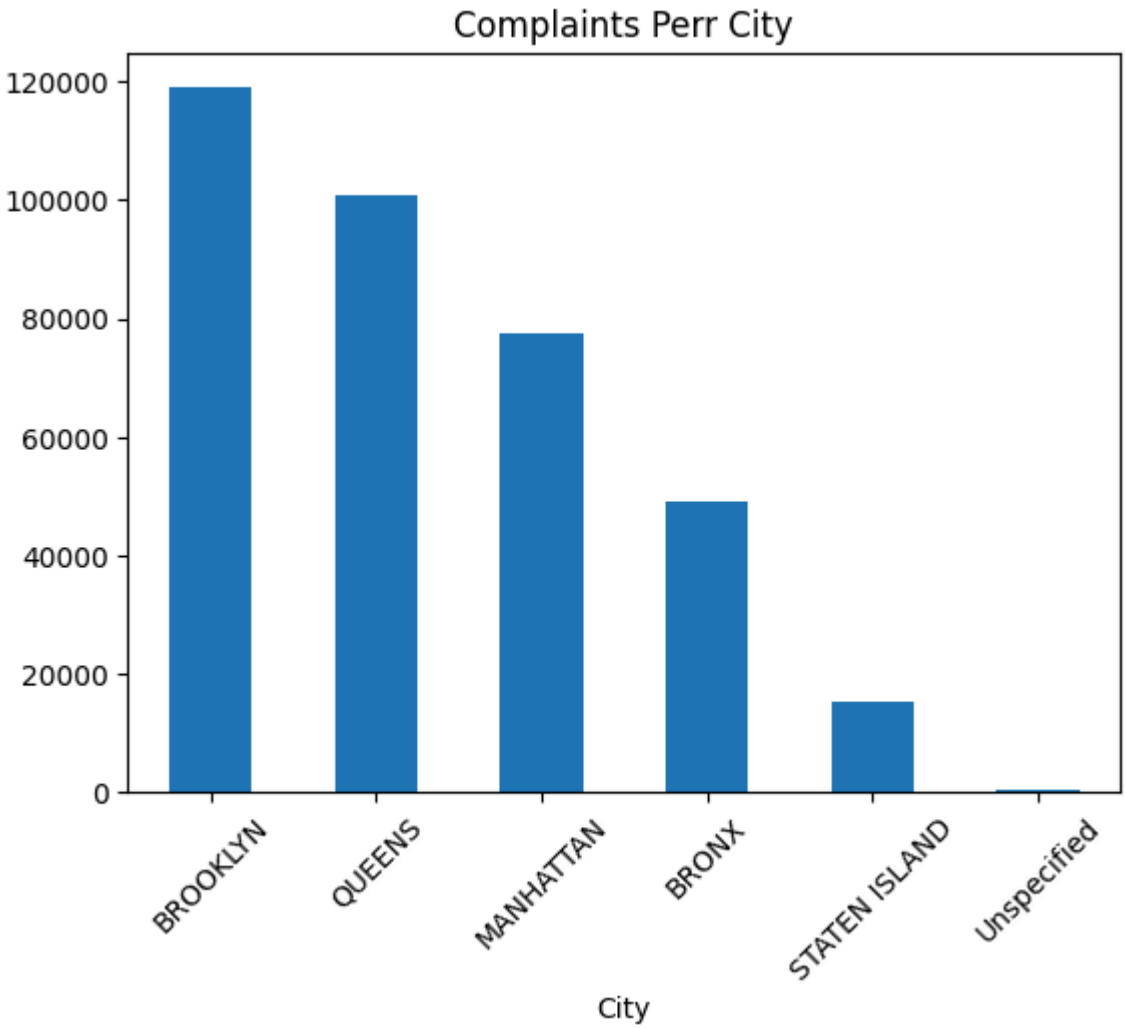
Out[14]:
```
Complaint Type    0
City              0
dtype: int64
```

2.3.5 Impute the NA value with Unknown City

In [15]:
```
df['City'] = df['City'].fillna("Unknown City")
```

2.3.6 Draw a frequency plot for the complaints in each city

In [16]:
```
df['City'].value_counts().plot(kind = 'bar')
plt.title("Complaints Perr City")
plt.xticks(rotation= 45)
plt.show()
```

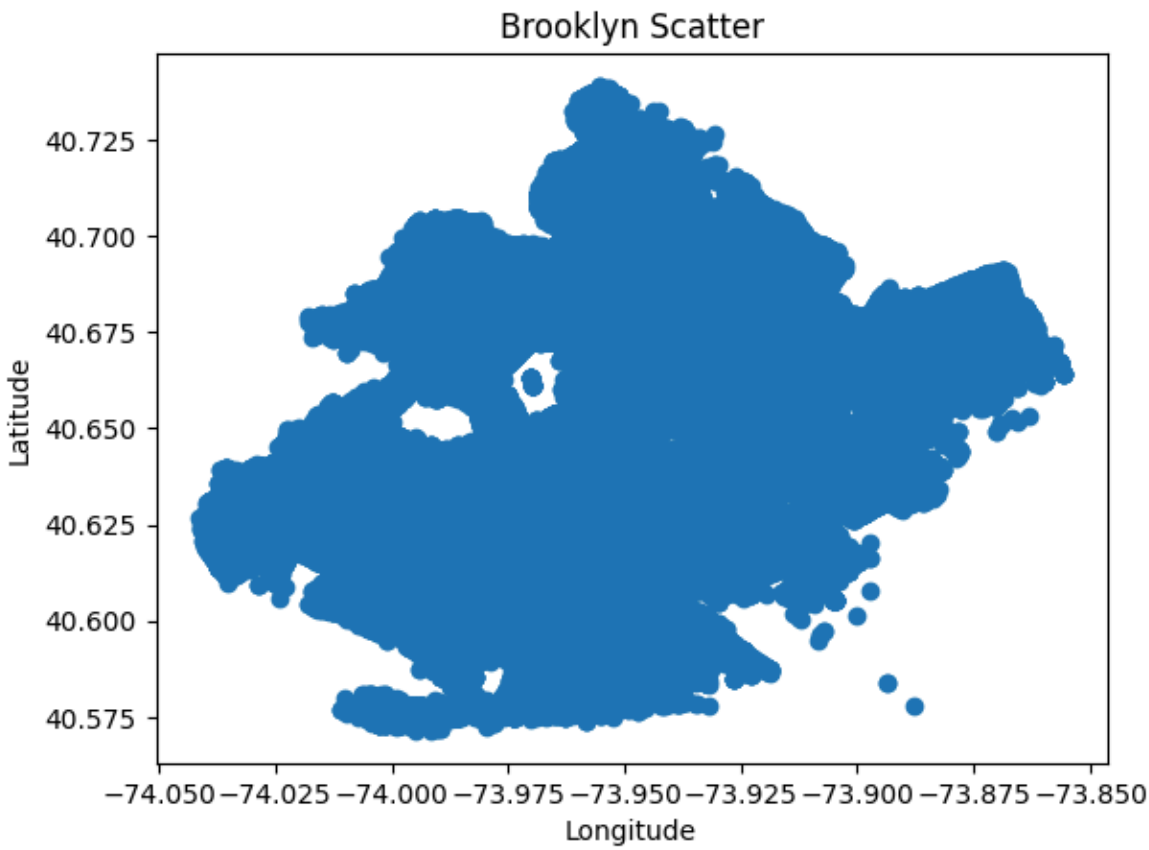


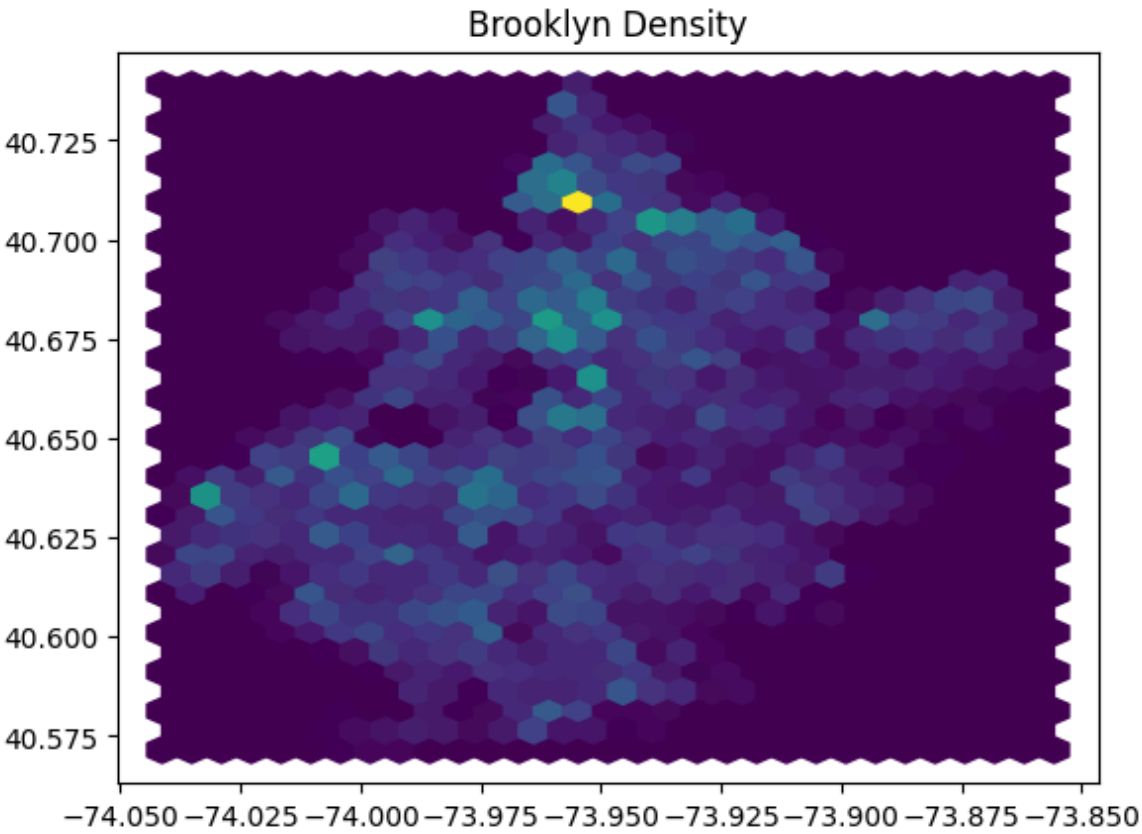
2.3.7 Brooklyn Scatter + Hexbin

In [17]:
```
# Filtering the dataframe for Brooklyn
brooklyn = df[df['Borough'] == 'BROOKLYN']

# --- Scatter Plot ---
plt.scatter(brooklyn['Longitude'], brooklyn['Latitude'])
plt.title("Brooklyn Scatter")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.show()

# --- Hexbin Plot ---
plt.hexbin(brooklyn['Longitude'], brooklyn['Latitude'], gridsize=30)
plt.title("Brooklyn Density")
plt.show()
```

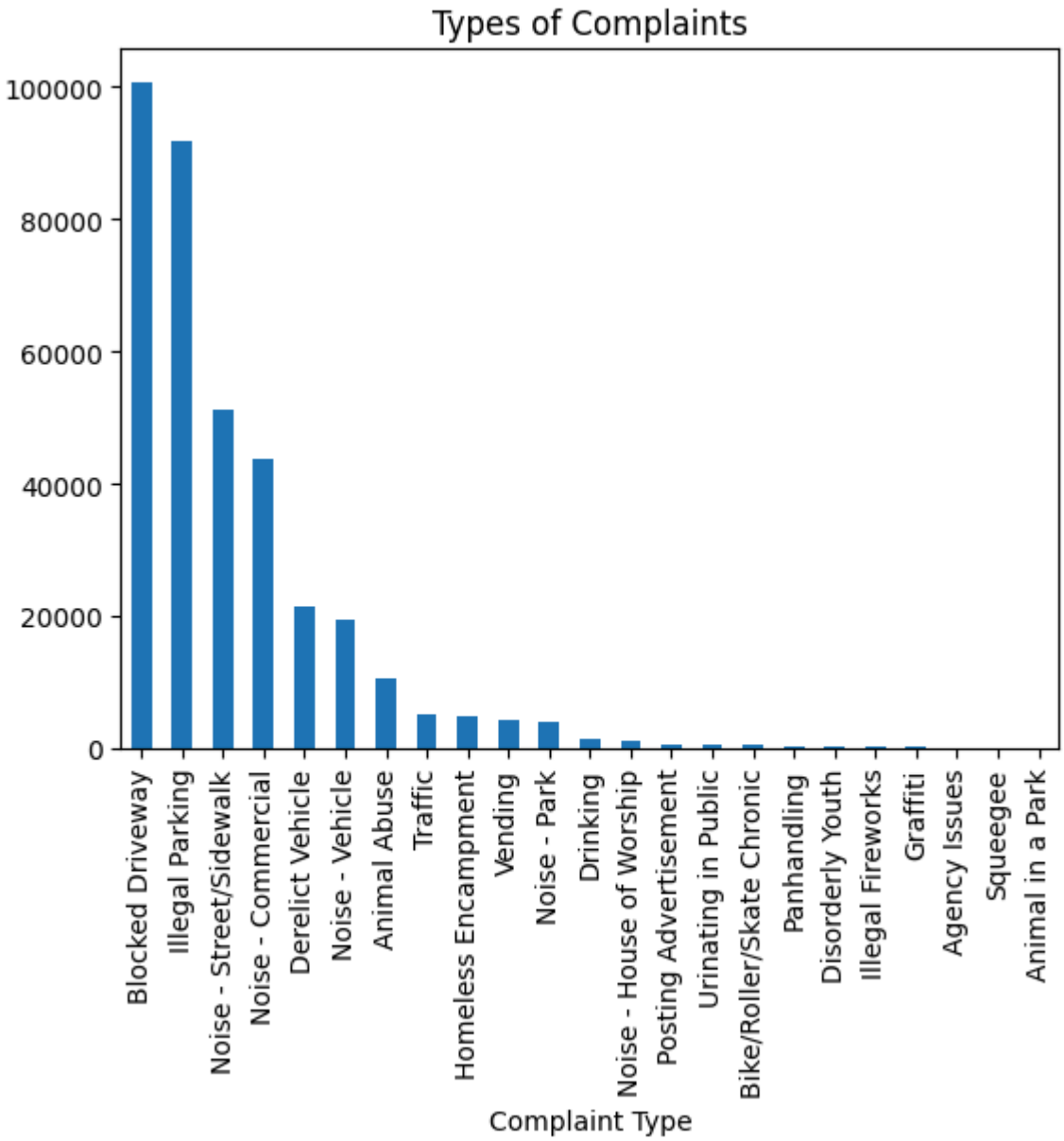




STEP 3: Find Major Complaint Types

3.1 Bar Graph(Complaint Types)

In [18]:
```
df['Complaint Type'].value_counts().plot(kind='bar')
plt.title("Types of Complaints")
plt.xticks(rotation = 90)
plt.show()
```



3.2 Check the frequency of various types of complaints for New York City

In [19]:
```
df[df['City'] == 'NEWYORK']['Complaint Type'].value_counts()
```

Out[19]:
```
Series([], Name: count, dtype: int64)
```

3.3 Top 10 Complaints

In [20]:
```
df['Complaint Type'].value_counts().head(10)
```

Out[20]:
```
Complaint Type
Blocked Driveway           100624
Illegal Parking             91716
Noise - Street/Sidewalk     51139
Noise - Commercial          43751
Derelict Vehicle            21518
Noise - Vehicle             19301
Animal Abuse                10530
Traffic                      5196
Homeless Encampment          4879
Vending                      4185
Name: count, dtype: int64
```

3.4 Complaints in Each City

In [21]:
```
pd.crosstab(df['City'],df['Complaint Type'])
```

Out[21]:

Complaint Type	Agency Issues	Animal Abuse	Animal in a Park	Bike/Roller/Skate Chronic	Blocked Driveway	Derelict Vehicle	Disorder You
City							
BRONX	0	1971	0	22	17062	2402	6
BROOKLYN	0	3191	0	124	36445	6258	7
MANHATTAN	0	1941	0	254	2706	694	8
QUEENS	0	2640	0	63	41481	9917	6
STATEN ISLAND	0	786	0	10	2845	2184	2
Unspecified	8	1	1	2	85	63	

6 rows × 23 columns

3.5 Create a DataFrame, df_new

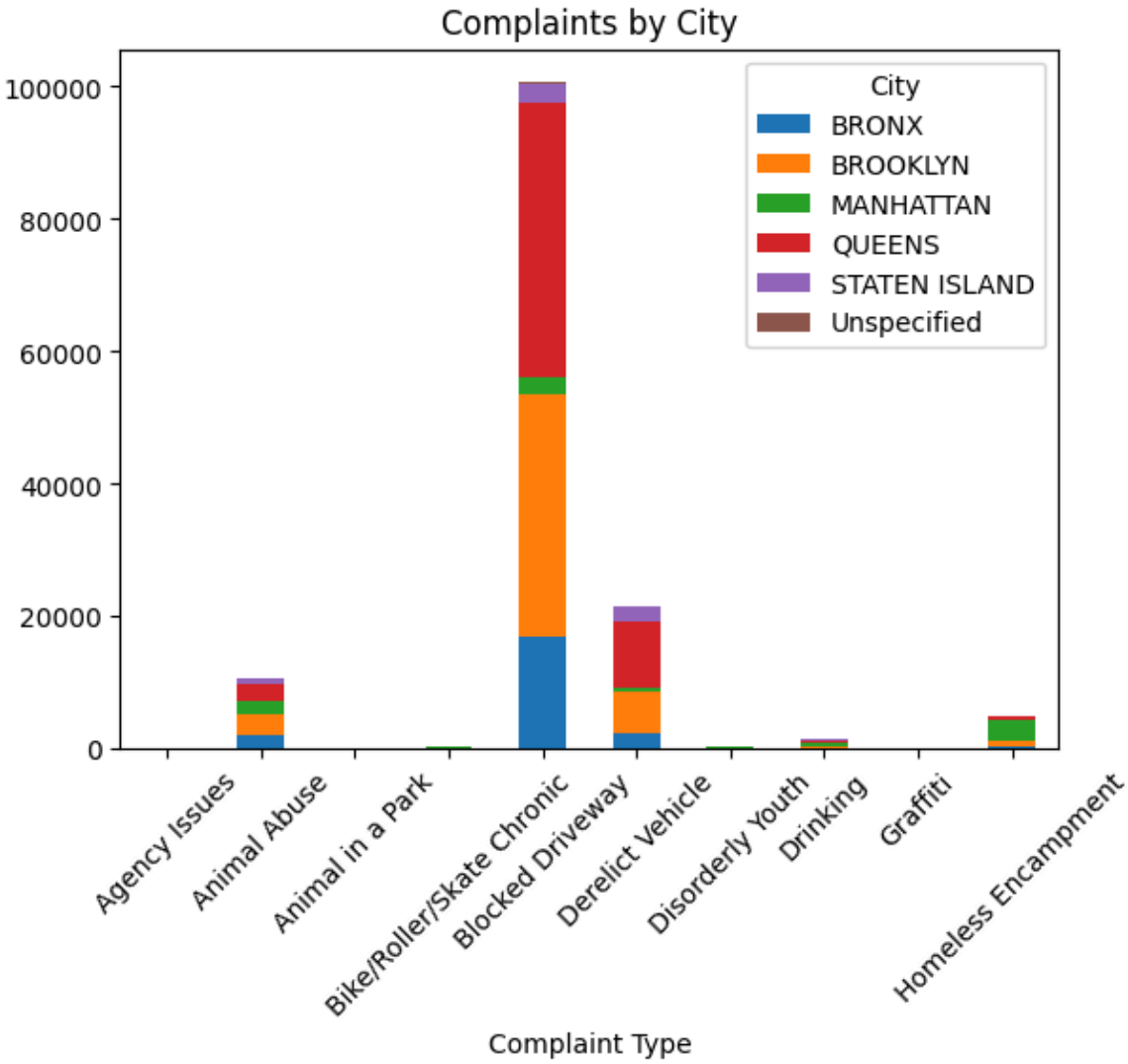
In [23]:
```
df_new = pd.pivot_table(
    df,
    index='Complaint Type',
    columns = 'City',
    aggfunc= 'size',
    fill_value= 0
)
```

STEP 4: Visualization

4.1 Multi-Color chart

In [24]:
```
df_new.head(10).plot(kind = 'bar', stacked = True)

plt.title("Complaints by City")
plt.xticks(rotation=45)
plt.show()
```



4.2 Sort By Avg Closing Time

In [25]:
```
avg_time = df.groupby(['Complaint Type', 'City'])['Request_Closing_Time'].mean()

avg_time = avg_time.sort_values()

avg_time.head()
```

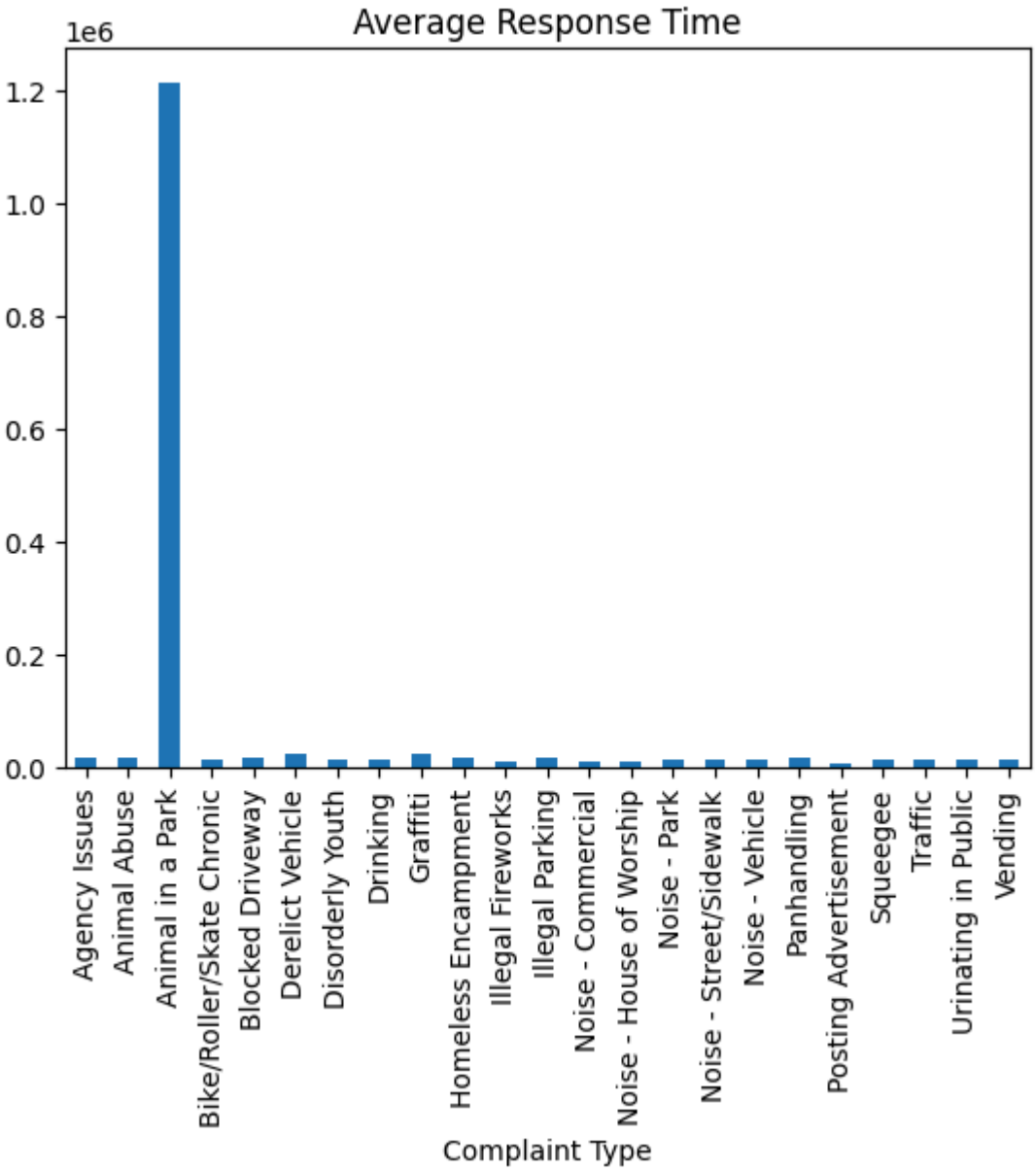
Out[25]:
```
Complaint Type            City
Posting Advertisement     STATEN ISLAND    5565.486434
Illegal Fireworks         MANHATTAN        7290.000000
Noise - House of Worship  MANHATTAN        8009.207207
Disorderly Youth          MANHATTAN        8133.419753
Illegal Fireworks         QUEENS           8358.105263
Name: Request_Closing_Time, dtype: float64
```

STEP 5: See whether the average response time across different complaint types is similar (overall)

5.1 Visualize the average of Request_Closing_Time

In [26]:
```
df.groupby('Complaint Type')['Request_Closing_Time'].mean().plot(kind='bar')

plt.title("Average Response Time")
plt.xticks(rotation = 90)
plt.show()
```



STEP 6: Statistical Analysis(p-value)

In [27]:
```
from scipy.stats import f_oneway

groups = df.groupby('Complaint Type')['Request_Closing_Time'].apply(list)

anova_result = f_oneway(*groups)

print(anova_result)
```

```
F_onewayResult(statistic=np.float64(565.261570041763), pvalue=np.float64(0.0))
```

STEP 7: Kruskal_Wallis Test

In [28]:
```
from scipy.stats import kruskal

kruskal_result = kruskal(*groups)

p = kruskal_result.pvalue

if p < 0.05:
    print("Reject H0: One or more distributions are not equal")
else:
    print("Fail to Reject H0: All distributions are equal")
```

```
Reject H0: One or more distributions are not equal
```

OR (OPTIONAL)

7.1 Fail To Reject HO:

In [32]:
```
if p >= 0.05:
    print("Fail to Reject H0: All sample distributions are equal")
```

7.2 Reject HO:

In [30]:
```
if p < 0.05:
    print("Reject H0: One or more distributions are not equal")
```

```
Reject H0: One or more distributions are not equal
```

STEP 8: Final Observations

In [29]:
```
# Most common complaint
most_common = df['Complaint Type'].value_counts().idxmax()

# City with max complaints
top_city = df['City'].value_counts().idxmax()

# Complaint with max response time
slowest = df.groupby('Complaint Type')['Request_Closing_Time'].mean().idxmax()

print("Most Common Complaint:", most_common)
```

```
print("Top City:", top_city)
print("Slowest Complaint Type:", slowest)
```

```
Most Common Complaint: Blocked Driveway
Top City: BROOKLYN
Slowest Complaint Type: Animal in a Park
```

OBSERVATIONS :

1.The dataset contains multiple types of complaints across different cities.

2.The most frequent complaints are related to noise, illegal parking, and blocked driveways.

3.New York City has the highest number of complaints compared to other cities.

4.The average response time varies depending on the type of complaint.

5.Some complaints are resolved quickly, while others take longer time.

6.Based on the Kruskal-Wallis test, there is a significant difference in response times across complaint types (if p-value < 0.05).

In [ ]:

```
```